فصل ۲۶: تقاطع‌های معماری

تاکنون در این کتاب نشان داده‌ایم که چگونه ویژگی‌های حیاتی‌ای را که یک معماری باید پشتیبانی کند شناسایی کنید، چگونه مناسب‌ترین سبک معماری را برای آن ویژگی‌ها و برای مسئلهٔ کسب‌وکار برگزینید، چگونه تصمیمات معماری اثربخش بگیرید و چگونه تیم‌های توسعه را در پیاده‌سازی معماری رهبری و راهنمایی کنید. با این حال، برای آنکه معماری کار کند، باید با دیگر جنبه‌های محیط فنی و کسب‌وکاری نیز همسو باشد. ما این همسویی‌ها را **تقاطع‌های معماری** `(intersections of architecture)` می‌نامیم.

در این فصل، چند تقاطع مهم را بررسی می‌کنیم که هنگام طراحی یا اعتبارسنجی یک معماری نرم‌افزار پدید می‌آیند:

- **پیاده‌سازی** `(Implementation)`: آیا پیاده‌سازی با دغدغه‌های معماری پیرامون ویژگی‌های عملیاتی، محدودیت‌های معماری و ساختار درونی معماری همسوست؟
- **زیرساخت** `(Infrastructure)`: آیا زیرساخت و شیوهٔ استقرار معماری با دغدغه‌های عملیاتی معماری، مانند مقیاس‌پذیری، پاسخ‌گویی، تحمل خطا و در دسترس‌بودن، همسوست؟
- **توپولوژی‌های داده** `(Data topologies)`: یکی از همسویی‌هایی که به‌طور گسترده نادیده گرفته می‌شود، همسوییِ میان معماری از یک سو و توپولوژی‌های داده و نوع داده از سوی دیگر است. توپولوژی داده (تک‌پارچه، پایگاه‌های دادهٔ حوزه‌ای و پایگاه‌داده به ازای هر سرویس) باید به‌درستی با سبک معماری همسو باشد تا سیستم کار کند.
- **رویه‌های مهندسی** `(Engineering practices)`: آیا شیوه‌ای که تیم توسعه نرم‌افزار را می‌سازد، نگهداری می‌کند و می‌آزماید با معماری متناظر آن جور است؟ آیا خط لولهٔ استقرار با سبک معماری جور است؟
- **توپولوژی‌های تیم** `(Team topologies)`: شیوهٔ سازمان‌دهی تیم‌ها می‌تواند اثر چشمگیری بر معماری داشته باشد و برعکس. اگر ساختار تیم به‌درستی با معماری همسو نباشد، تیم‌های توسعه معمولاً دچار دشواری می‌شوند و حتی ساده‌ترین تغییرات را نیز چالش‌برانگیز می‌یابند.
- **یکپارچه‌سازی سیستم‌ها** `(Systems integration)`: معماری باید با چه سیستم‌ها یا سرویس‌های دیگری ارتباط برقرار کند؟ بی‌توجهی به این تقاطع خاص می‌تواند از نظر نگهداری‌پذیری، قابلیت اطمینان و ویژگی‌های عملیاتی مانند مقیاس‌پذیری، پاسخ‌گویی و در دسترس‌بودن، پیامدهای ویرانگری داشته باشد.
- **سازمان** `(The enterprise)`: آیا معماری با چارچوب‌ها، رویه‌ها، اصول راهنما و استانداردهای جاری در سراسر سازمان و بنگاه همسوست؟
- **محیط کسب‌وکار** `(The business environment)`: آیا معماری به‌درستی با محیط کسب‌وکار و حوزهٔ مسئله همسوست؟ معماران بارها این تقاطع مهم را نادیده می‌گیرند و در نتیجه معماری از برآوردن اهداف یا نیازهای کسب‌وکار باز می‌ماند.
- **هوش مصنوعی مولد** `(Generative AI)`: استفادهٔ روزافزون از مدل‌های زبانی بزرگ (large language models یا LLMs) چه اثری بر معماری دارد؟ با بهره‌گیری شرکت‌های بیشتر از هوش مصنوعی مولد در سیستم‌هایشان، این تقاطع به‌سرعت در حال تبدیل شدن به تقاطعی مهم است.

بخش‌های بعدی این تقاطع‌ها را با جزئیات بیشتری شرح می‌دهند.

معماری و پیاده‌سازی

قانون اول معماری نرم‌افزار، رایج‌ترین پاسخ معماران نرم‌افزار به هر پرسشی نیز هست: «بستگی دارد.» شاید دومین پاسخ رایج این باشد: «این یک جزئیات پیاده‌سازی است.» وقتی یک معماری نرم‌افزار به اهدافش نمی‌رسد، اغلب همین پاسخ دوم مقصر است.

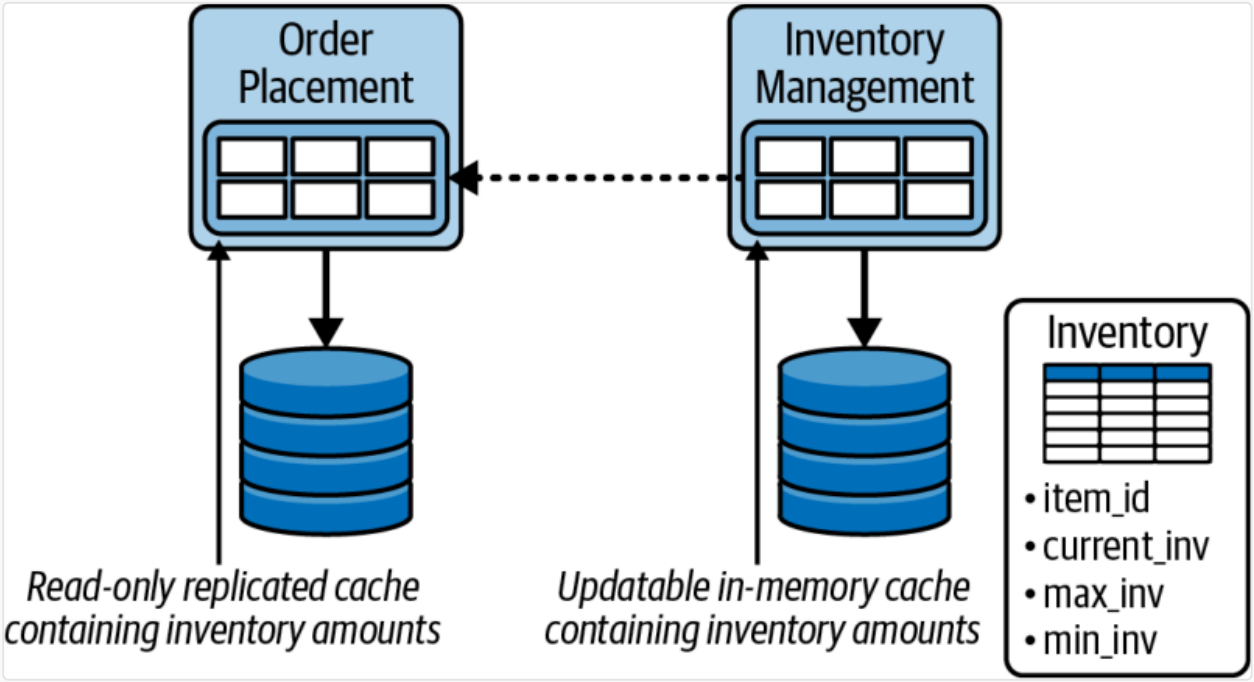
برای آنکه معماری به‌درستی کار کند، **پیاده‌سازی** آن، یعنی کد منبعش، باید با طراحی‌اش همسو باشد تا به سه چیز بپردازد: دغدغه‌های عملیاتی معماری (مانند تحمل خطا، پاسخ‌گویی، مقیاس‌پذیری و مانند آن)، ساختار درونی و محدودیت‌ها. این بخش به ترتیب به هر یک از این سه می‌پردازد.

دغدغه‌های عملیاتی

دغدغه‌های عملیاتی `(operational concerns)` همان ویژگی‌های معماری‌اند که در بخش نخست کتاب بر آن‌ها تمرکز کردیم؛ ویژگی‌هایی که پایهٔ هر معماری نرم‌افزار را تشکیل می‌دهند و معماری باید برای حل مسئلهٔ کسب‌وکارِ پیش رو از آن‌ها پشتیبانی کند. ویژگی‌های معماری نیروی محرکهٔ تصمیمات معماری‌اند (چنان‌که در فصل ۲۱ بررسی شد).

پس ناهمسو شدنِ معماری و پیاده‌سازی در شیوهٔ پرداختن به دغدغه‌های عملیاتی سیستم به چه معناست؟ فرض کنید معماری هستید که روی یک سیستم ثبت سفارش جدید کار می‌کند؛ سیستمی که باید از چند هزار تا نیم میلیون مشتری هم‌زمان را پشتیبانی کند. بر پایهٔ رتبه‌بندی ستاره‌ای در «ویژگی‌های سبک» در صفحهٔ ۳۵۴ از فصل ۱۸، سبک معماری میکروسرویس‌ها را برمی‌گزینید که با توجه به نیازهای بالای این سیستم به مقیاس‌پذیری و کشسانی مناسب است. با این حال، در جریان **پیاده‌سازی**، تیم توسعه درمی‌یابد که چون زمینه‌های محدودِ سرویس‌ها بسیار تنگ شکل گرفته‌اند (به «زمینهٔ محدود» در صفحهٔ ۳۳۱ از فصل ۱۸ بنگرید)، سرویس `Order Placement` نمی‌تواند مستقیماً به پایگاه‌دادهٔ موجودی دسترسی داشته باشد. در عوض، باید سرویس `Inventory` را به‌صورت هم‌گام فراخوانی کند تا موجودی فعلی هر کالایی را که مشتری به خرید آن علاقه دارد به دست آورد. این فراخوانی هم‌گام نه‌تنها دو سرویس را به‌شدت به هم کوپل می‌کند، بلکه پاسخ‌گویی سیستم را به‌شدت کند می‌کند.

تیم توسعه تصمیم می‌گیرد میان این سرویس‌ها از یک **کش تکثیرشدهٔ درون‌حافظه‌ای** `(in-memory replicated cache)` استفاده کند، چنان‌که در شکل ۲۶-۱ نشان داده شده است: داده در **حافظهٔ داخلی** `(internal memory)` هر نمونهٔ سرویس قرار دارد و از راه کش‌گذاری، در پشت صحنه همواره هم‌گام نگه داشته می‌شود. از جمله محصولات کش‌گذاری برای این منظور می‌توان به `Apache Ignite` و `Hazelcast` اشاره کرد. در اینجا، سرویس `Inventory` یک کش درون‌حافظه‌ایِ نوشتنی دارد که شناسه‌های کالا و شمار موجودی فعلی را در بر می‌گیرد؛ هر نمونه از سرویس `Order Placement` نیز نسخه‌ای تکثیرشده و فقط‌خواندنی از آن کش را در حافظهٔ داخلی خود دارد. استفاده از کش تکثیرشدهٔ درون‌حافظه‌ای سرویس‌ها را از هم جدا می‌کند و پاسخ‌گویی را **به‌طور چشمگیری** بهبود می‌بخشد.



شکل ۲۶-۱. تیم توسعه تصمیم گرفت میان سرویس‌ها از کش‌گذاری تکثیریِ درون‌حافظه‌ای استفاده کند و در نتیجه، هنگام مقیاس‌دهی سرویس‌ها، وضعیت‌های کمبود حافظه پدید آمد

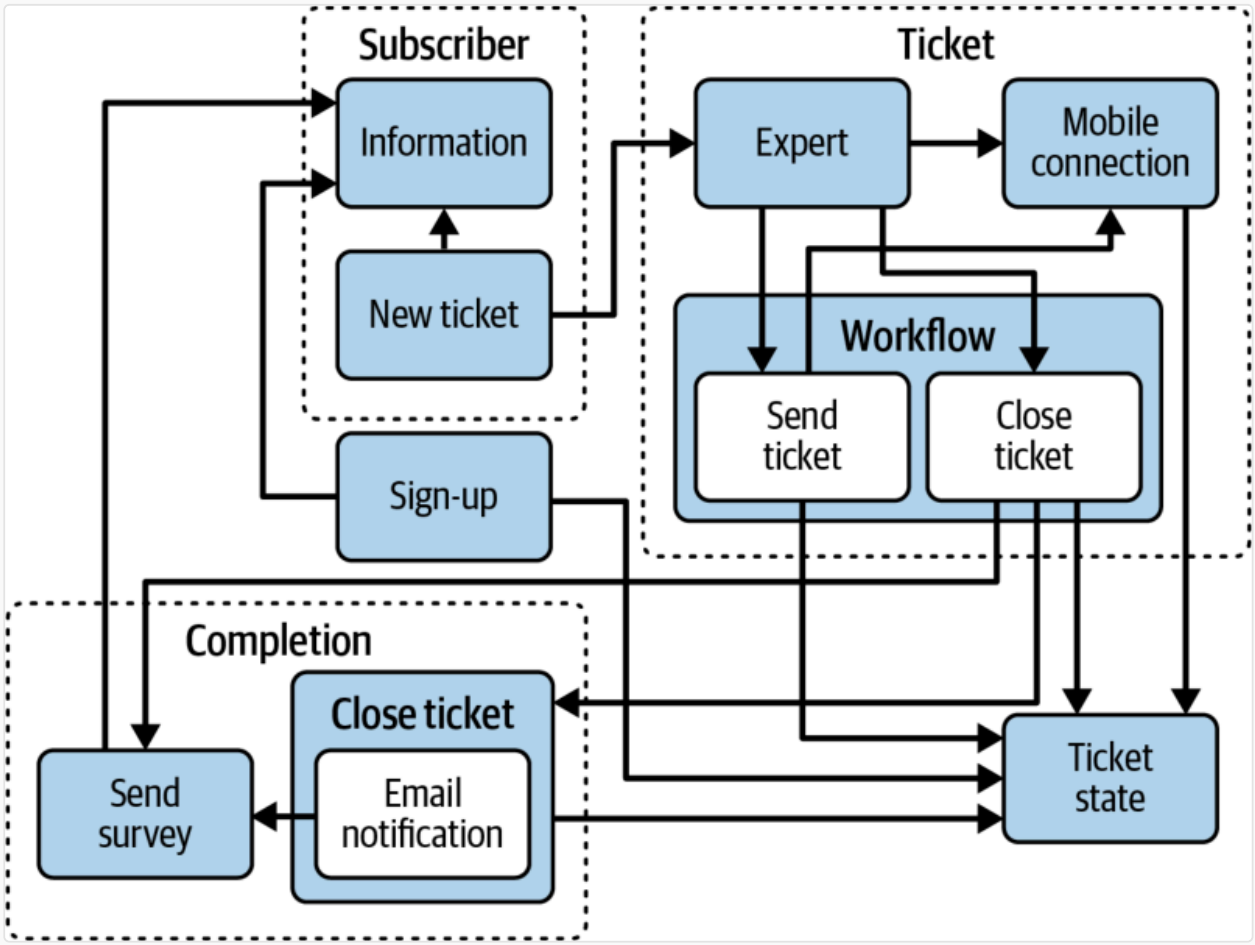
پس از انتشار در محیط تولید، با افزایش شمار کاربران هم‌زمان، نمونه‌های بیشتری از هر سرویس برای رسیدگی به بار لازم می‌شود. وقتی بار به حدود ۸۰,۰۰۰ مشتری هم‌زمان می‌رسد، سیستم از کار می‌افتد، زیرا نیاز کش داخلی به حافظه بیش از حد بالاست و در همهٔ ماشین‌های مجازی وضعیت کمبود حافظه `(out-of-memory)` پدید می‌آورد.

در این سناریو، معماری و پیاده‌سازی آن ناهمسوست. در حالی که معماری بر پشتیبانی از سطوح بالای **مقیاس‌پذیری** و **کشسانی** تمرکز دارد، پیاده‌سازی بر **پاسخ‌گویی** و **جداسازی سرویس‌ها** تمرکز دارد. هر دو تیم تصمیم‌های خوبی گرفتند، اما در خدمت اهدافی متفاوت.

یکپارچگی ساختاری

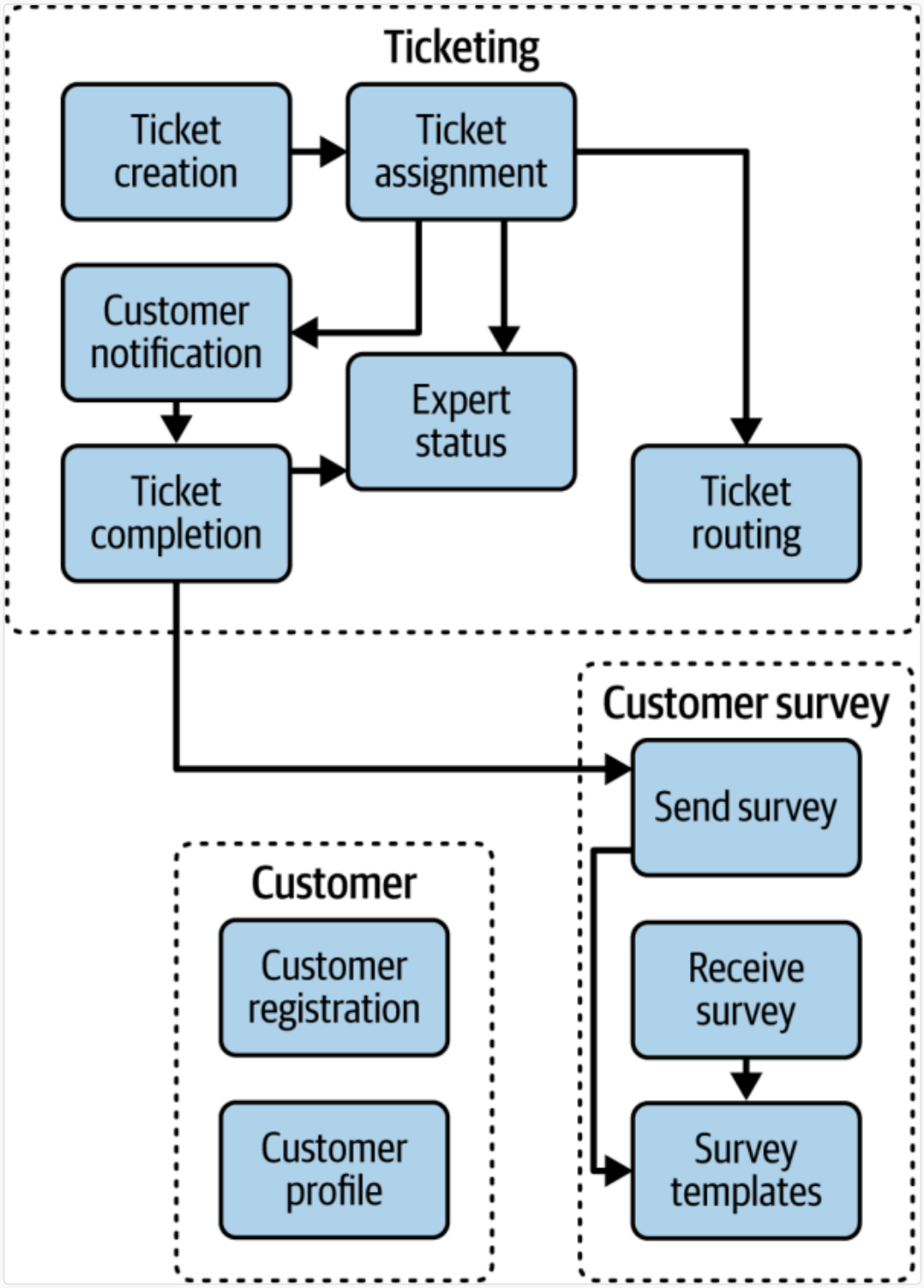
در فصل ۸ آموختید که مؤلفه‌های منطقی بلوک‌های سازندهٔ هر سیستم‌اند و **معماری منطقی** `(logical architecture)` آن را تشکیل می‌دهند. این مؤلفه‌ها معمولاً از راه ساختار پوشه‌ها در مخزن کد منبع (یا بسته به زبان برنامه‌نویسی، فضاهای نام) بازنمایی می‌شوند. چون معماری منطقی توصیف می‌کند که سیستم چگونه کار می‌کند و کدام بخش‌های سیستم با کدام بخش‌های دیگر تعامل دارند، حیاتی است که ساختار کد منبع با ساختار معماری منطقی جور باشد.

بدون راهنمایی، دانش و حاکمیت مناسب، توسعه‌دهندگان به‌آسانی معماری منطقی سیستم را نادیده می‌گیرند و بی‌توجه به اثر کارشان بر یکپارچگی سیستم، ساختار پوشه‌ها و فضاهای نام را به دلخواه خود می‌سازند. این ناهمسویی به معماری‌هایی می‌انجامد که نگهداری، آزمون و استقرارشان دشوار است و در نتیجه کم‌اطمینان‌تر می‌شوند و تکامل یا سازگار شدنشان با قابلیت‌های تازه سخت‌تر می‌شود؛ مانند معماری منطقی‌ای که در شکل ۲۶-۲ نشان داده شده است.



شکل ۲۶-۲. نمونه‌ای از یک معماری منطقیِ داخلی که فاقد حاکمیت و همسویی است

برای اطمینان از اینکه ساختار کد منبع با معماری منطقی جور است، استفاده از ابزارهای حاکمیت خودکار را توصیه می‌کنیم؛ مانند `ArchUnit` برای سکوی `Java`، `ArchUnitNet` و `NetArchTest` برای سکوی `.NET`، `PyTestArch` برای `Python`، یا `TSArch` برای `TypeScript` و `JavaScript`. این ابزارهای خودکار، همراه با ارتباط و همکاری خوب میان معمار و تیم توسعه، پیاده‌سازی‌هایی پدید می‌آورند که به‌درستی با معماری همسو هستند، چنان‌که در شکل ۲۶-۳ نشان داده شده است.



شکل ۲۶-۳. نمونه‌ای از یک معماری منطقیِ داخلی که بر پایهٔ حاکمیت و همسویی مناسب شکل گرفته است

شکل ۲۶-۲ را با شکل ۲۶-۳ مقایسه کنید. توجه کنید که معماری شکل ۲۶-۳ تا چه اندازه از معماری ناهمسوی شکل ۲۶-۲ نگهداری‌پذیرتر، آزمون‌پذیرتر، استقرارپذیرتر، قابل‌اطمینان‌تر، سازگارپذیرتر و توسعه‌پذیرتر است. این مقایسه اهمیت این نوع همسویی در پیاده‌سازی را نشان می‌دهد.

محدودیت‌های معماری

محدودیت `(constraint)` قاعده یا اصلی حاکم است که نوعی محدودیت را در معماری توصیف می‌کند (مانند محدود کردن ارتباطات فقط به `REST` یا استفاده از نوع مشخصی از پایگاه‌داده) و برای رسیدن معماری به اهدافش لازم است. اگر پیاده‌سازی سیستم به محدودیت‌های آن پایبند نباشد، معماری شکست می‌خورد. از این رو، بخشی از کار معمار نرم‌افزار شناسایی و انتقال **محدودیت‌های** معماری است.

برای روشن کردن منظورمان از این تقاطع خاص، کسب‌وکاری را در نظر بگیرید که می‌کوشد سیستم جدیدی را با بودجه‌ای بسیار محدود و مهلتی فشرده منتشر کند. این کسب‌وکار انتظار تغییرات ساختاری فراوانی در پایگاه‌داده دارد و لازم است این تغییرات هرچه سریع‌تر انجام شوند. در این حالت، معماری لایه‌ای سنتی (فصل ۱۰ را ببینید) به سبب سادگی، صرفه‌جویی در هزینه و افراز فنی، گزینهٔ بسیار مناسبی است. چون این سبک لایه‌ها را از هم جدا می‌کند، تغییرات پایگاه‌داده را می‌توان تنها به یک لایه محدود کرد و انجامشان را آسان‌تر و سریع‌تر ساخت.

برای آنکه معماری لایه‌ای در این مسئلهٔ کسب‌وکارِ خاص کار کند، معمار باید محدودیت‌های زیر را تعریف کند:

- همهٔ منطق پایگاه‌داده باید در لایهٔ `Persistence` قرار گیرد.
- لایهٔ `Presentation` نمی‌تواند مستقیماً به لایهٔ `Persistence` دسترسی داشته باشد و باید، حتی برای پرس‌وجوهای ساده، از همهٔ لایه‌ها عبور کند.

این محدودیت‌ها لازم‌اند تا از پراکنده شدن منطق پایگاه‌داده در سراسر معماری جلوگیری شود و تغییرات در ساختار فیزیکی پایگاه‌داده (مانند حذف یک جدول یا تغییر نام یک ستون) بر هیچ کدی بیرون از لایهٔ `Persistence` اثر نگذارد.

اکنون فرض کنید توسعه‌دهندگان رابط کاربری تصمیم می‌گیرند که فراخوانی مستقیم پایگاه‌داده سریع‌تر است و معماری را به جای آن به همین شیوه پیاده‌سازی می‌کنند. افزون بر این، توسعه‌دهندگان بک‌اند درمی‌یابند که اگر منطق `Business` و منطق پایگاه‌داده کنار هم باشند، نگهداری و آزمون کد بسیار آسان‌تر خواهد بود؛ پس آن‌ها نیز محدودیت‌ها را نادیده می‌گیرند و این دغدغه‌ها را در لایهٔ کسب‌وکارِ معماری به هم کوپل می‌کنند. این پیاده‌سازی با محدودیت‌های معماری همسو نیست؛ یعنی تغییرات پایگاه‌داده بر همهٔ کدهای هر لایه اثر می‌گذارند، بیش از حد زمان می‌برند و سیستم به اهداف کسب‌وکار نخواهد رسید.

ابزارهای معماری برای حاکمیت بر محدودیت‌های معماری نیز سودمندند.

معماری و زیرساخت

دامنهٔ معماری نرم‌افزار در دو دههٔ گذشته گسترش یافته و مسئولیت‌ها و دیدگاه‌های هرچه بیشتری را در بر گرفته است. در حدود میانهٔ دههٔ ۲۰۰۰، رابطهٔ معمول میان معماری و عملیات `(operations)` قراردادی و رسمی و همراه با دیوان‌سالاری فراوان بود. بیشتر شرکت‌ها برای پرهیز از پیچیدگیِ میزبانی عملیاتِ خود، عملیات را به یک شخص ثالث برون‌سپاری می‌کردند و برای زمان فعالیت، مقیاس، پاسخ‌گویی و دیگر ویژگی‌های مهم توافق‌نامه‌های سطح خدمت

(SLA) می‌بستند. اما امروزه سبک‌های معماری‌ای مانند میکروسرویس‌ها آزادانه از ویژگی‌هایی بهره می‌گیرند که پیش‌تر صرفاً عملیاتی بودند. برای مثال، **مقیاس‌پذیری کشسان** (elastic scale) زمانی در درون معماری‌های مبتنی بر فضا ساخته می‌شد (فصل ۱۶ را ببینید)، اما امروز میکروسرویس‌ها از راه همکاری تنگاتنگ‌تر میان معماران و DevOps، با دردسر کمتری از پسِ آن برمی‌آیند.

> **تاریخچه: Pets.com چگونه مقیاس‌پذیری کشسان را به ما داد (History: How Pets.com Gave Us Elastic Scale)**
>
> مردم اغلب گمان می‌کنند توانایی‌های فنیِ کنونی ما (مانند مقیاس‌پذیری کشسان) روزی به دست توسعه‌دهنده‌ای زیرک ابداع شده‌اند. اما در واقعیت، بهترین ایده‌ها اغلب از دل درس‌های تلخ زاده می‌شوند. Pets.com نمونه‌ای از همان روزهای نخست است. این وب‌سایت تجارت الکترونیک در حدود سال ۱۹۹۸ پدید آمد، به این امید که به Amazon.com لوازم حیوانات خانگی تبدیل شود. بخش بازاریابیِ درخشان آن نماد تبلیغاتیِ گیرایی پدید آورد: عروسکی جورابی با یک میکروفون که حرف‌های گستاخانه می‌زد. این نماد به یک ابرستاره تبدیل شد و در رژه‌ها و رویدادهای ورزشیِ ملی در برابر عموم ظاهر می‌شد.
>
> متأسفانه، مدیریت Pets.com ظاهراً همهٔ پول را صرف نماد تبلیغاتی کرد، نه زیرساخت. وقتی سفارش‌ها سرازیر شدند، آمادگی نداشتند. وب‌سایت کند بود، تراکنش‌ها گم می‌شدند، تحویل‌ها به تأخیر می‌افتادند... کمابیش بدترین سناریوی ممکن بود. اندکی پس از هجومِ فاجعه‌بارِ کریسمس، Pets.com تعطیل شد و تنها دارایی ارزشمندِ باقی‌مانده‌اش، یعنی همان نماد تبلیغاتی، را فروخت.
>
> آنچه Pets.com به آن نیاز داشت **مقیاس‌پذیری کشسان** بود: توانایی راه‌اندازی نمونه‌های بیشتری از منابع، هر زمان که لازم شوند. ارائه‌دهندگان ابر اکنون این قابلیت را به‌صورت کالایی همگانی عرضه می‌کنند، اما شرکت‌های تجارت الکترونیکِ نخستین باید زیرساخت خود را مدیریت می‌کردند و بسیاری از آن‌ها قربانی پدیده‌ای ناشنیده شدند: موفقیتِ بیش از حد می‌تواند یک کسب‌وکار را از پا درآورد. سقوط Pets.com و دیگر داستان‌های ترسناکِ مشابه، معماران را واداشت هنگام طراحی معماری‌های نرم‌افزار به چنین تقاطع‌هایی توجه بیشتری کنند.

تقاطع معماری و زیرساخت اهمیت دارد، زیرا ویژگی‌های عملیاتیِ معماری را تسهیل می‌کند. اینکه یک **معماری** می‌تواند از مقیاس‌پذیری بالا پشتیبانی کند، به این معنا نیست که چنین خواهد کرد؛ اگر **زیرساختِ** متناظر از آن پشتیبانی نکند، چنین نخواهد شد (چنان‌که Pets.com نشان داد). ما در محل مشتریان بارها دیده‌ایم که معماران و توسعه‌دهندگان به سبب شکست‌های معماری سرزنش شده‌اند، در حالی که علت واقعیِ آن شکست‌ها ناهمسویی میان معماری و زیرساخت بوده است.

در بیشتر موارد، این ناهمسویی به سبب کمبود ارتباط و همکاری میان معمار و مسئولان زیرساخت و عملیات رخ می‌دهد. معماران اغلب اثر زیرساخت را بر ویژگی‌هایی مانند مقیاس‌پذیری، پاسخ‌گویی، تحمل خطا، کارایی، در دسترس‌بودن، کشسانی و مانند آن درنمی‌یابند. همین ناهمسویی بود که حوزهٔ `DevOps` را پدید آورد.

سال‌های بسیار، بسیاری از شرکت‌ها عملیات را جدا از توسعهٔ نرم‌افزار می‌دانستند و اغلب برای صرفه‌جویی در هزینه آن را به یک شرکت ثالث برون‌سپاری می‌کردند. در دهه‌های ۱۹۹۰ و ۲۰۰۰، بسیاری از معماری‌ها به‌صورت تدافعی و بر پایهٔ این فرض طراحی می‌شدند که عملیات برون‌سپاری خواهد شد و در نتیجه از کنترل معماران بیرون خواهد بود. (برای نمونه‌ای خوب از این وضعیت، فصل ۱۶ را ببینید.) اما در میانهٔ دههٔ ۲۰۰۰، شرکت‌ها آزمودنِ شکل‌های تازه‌ای از معماری را آغاز کردند که بسیاری از دغدغه‌های عملیاتی را در خود جمع می‌کرد. برای مثال، سبک‌های معماریِ قدیمی‌تر مانند `SOA` ارکستراسیون‌محور برای پشتیبانی از قابلیت‌هایی مانند مقیاس‌پذیری و کشسانی به ابزارها و چارچوب‌های پرجزئیاتی نیاز داشتند که پیاده‌سازی را به‌شدت پیچیده می‌کرد. پس معماران معماری‌هایی ساختند که می‌توانستند مقیاس، کارایی، کشسانی و انبوهی از قابلیت‌های دیگر را **به‌صورت درونی** مدیریت کنند. پیامد جانبی این بود که این معماری‌ها به‌مراتب پیچیده‌تر شدند.

پدیدآورندگان سبک معماری میکروسرویس‌ها دریافتند که بهتر است دغدغه‌های عملیاتی را خودِ عملیات مدیریت کند. معماران با ایجاد رابطه‌ای همکارانه میان معماری و عملیات دریافتند که می‌توانند طراحی‌های خود را ساده کنند و به افراد عملیات تکیه کنند تا کارهایی را انجام دهند که در آن‌ها از همه بهترند. آن‌ها با عملیات هم‌دست شدند تا میکروسرویس‌ها را پدید آورند و بنیان چیزی را بگذارند که بعدها به جنبش `DevOps` تبدیل شد. هرچند `DevOps` کمک کرده است، این تقاطعِ معماری و زیرساخت همچنان برای بیشتر شرکت‌ها مسئله‌ساز است.

هرچند زیرساخت دیگر دغدغهٔ چندان بزرگی نیست، محیط‌های ابری همچنان ممکن است با معماری ناهمسو شوند. برای مثال، استقرار سرویس‌ها در چند منطقه `(region)` یا حتی چند ناحیهٔ دسترس‌پذیری `(availability zone)` می‌تواند مزایای کارایی و یکپارچگی دادهٔ کش‌های تکثیرشدهٔ درون‌حافظه‌ای و کش‌های توزیع‌شده را کاهش دهد یا حتی از میان ببرد. به همین ترتیب، هم‌مکان‌سازیِ `(co-locating)` سرویس‌ها، کانتینرها یا حتی Podهای `Kubernetes` روی یک ماشین مجازی کارایی را به‌طور چشمگیری افزایش می‌دهد، اما بر مقیاس‌پذیری، تحمل خطا، در دسترس‌بودن و کشسانی اثر منفی می‌گذارد.

همسو کردنِ معماری با زیرساخت نیازمند ارتباط و همکاری نزدیک میان معماران و اعضای تیم زیرساخت، یا حتی پذیرش رویه‌های `DevOps` است تا همهٔ ذی‌نفعان دغدغه‌های عملیاتیِ حیاتی را درک کنند. تنها در این صورت است که معماران می‌توانند به‌راستی از مزایای عملیاتیِ معماری‌ای که برگزیده‌اند بهره‌مند شوند؛ همان مزایایی که ما را به اعطای آن رتبه‌بندی‌های پنج‌ستارهٔ شگفت‌انگیز واداشت.

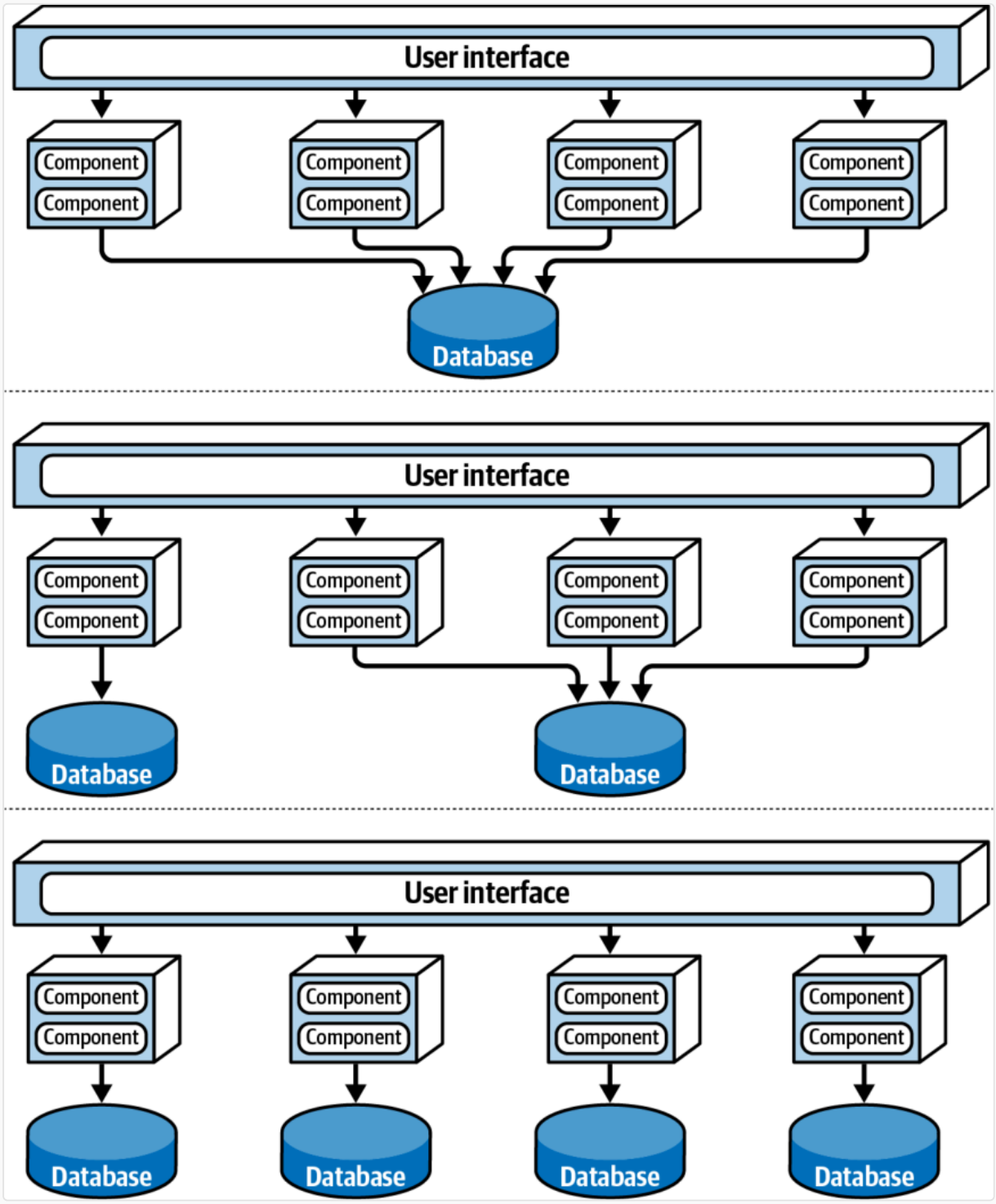
معماری و توپولوژی‌های داده

تقاطع معماری و توپولوژی‌های داده اغلب نادیده گرفته می‌شود. انتخاب نوع یا توپولوژی نادرستِ پایگاه‌داده می‌تواند به معماری آسیب بزند و بهترین ویژگی‌های معماری آن را خنثی کند. برای مثال، پایگاه‌داده‌های تک‌پارچه هرچند سازگاری داده و پشتیبانی تراکنشیِ خوبی فراهم می‌کنند، می‌توانند از مقیاس‌پذیری و تحمل خطا بکاهند. به همین ترتیب، توپولوژی‌های پایگاه‌دادهٔ توزیع‌شده هرچند در مقیاس‌پذیری و کنترل تغییر خوب‌اند، می‌توانند یکپارچگی داده، سازگاری داده و کارایی سیستم را کاهش دهند.

بخش‌های بعدی تقاطع معماری و توپولوژی‌های داده را شرح می‌دهند.

توپولوژی پایگاه‌داده

توپولوژی یک پایگاه‌داده، چنان‌که در فصل ۱۵ بررسی کردیم، به چگونگی پیکربندی پایگاه‌داده‌های فیزیکیِ آن در معماری اشاره دارد. سه توپولوژی پایه عبارت‌اند از: پایگاه‌دادهٔ تک‌پارچه، پایگاه‌داده‌های توزیع‌شدهٔ حوزه‌ای، و توپولوژی توزیع‌شدهٔ پایگاه‌داده به ازای هر سرویس. شکل ۲۶-۴ این انواع پایهٔ توپولوژی پایگاه‌داده را نشان می‌دهد.



شکل ۲۶-۴. انواع رایج توپولوژی‌های پایگاه‌داده

توپولوژی پایگاه‌داده باید با معماری همسو باشد تا درست کار کند. برای مثال، معماری‌های میکروسرویس معمولاً از الگوی پایگاه‌داده به ازای هر سرویس استفاده می‌کنند تا زمینهٔ محدودِ سخت‌گیرانه‌ای را حفظ کنند. بدون این همسوییِ مناسب، مهار تغییر برای معماران به‌شدت چالش‌برانگیز می‌شد و ویژگی‌های عملیاتی سیستم، مانند تحمل خطا، مقیاس‌پذیری، کشسانی، نگهداری‌پذیری، آزمون‌پذیری و استقرارپذیری، همگی آسیب می‌دیدند. با این حال، برخی سبک‌ها، مانند معماری سرویس‌محور (فصل ۱۴ را ببینید)، در مورد توپولوژی فیزیکی پایگاه‌داده انعطاف بیشتری دارند.

ویژگی‌های معماری

در بخش دوم این کتاب نشان دادیم که هر سبک معماری ابرتوانایی‌هایی `(superpowers)` دارد (با رتبهٔ ۴ تا ۵ ستاره) و نقطه‌ضعف‌هایی (۱ تا ۲ ستاره). انواع پایگاه‌داده نیز چنین‌اند؛ و مهم است که ابرتوانایی‌های معماریِ یک سیستم با ابرتوانایی‌های متناظرِ نوع پایگاه‌داده‌اش همسو شود. ما نویسندگان در کتاب دیگرمان، **`Software Architecture: The Hard Parts`**، ویژگی‌های شش نوع پایگاه‌داده را رتبه‌بندی کرده‌ایم: رابطه‌ای، کلید-مقدار `(key-value)`، سندمحور، ستونی `(columnar)`، گرافی `(graph)` و `NoSQL`. شاید به یاد داشته باشید که مقیاس‌پذیری و کشسانی ابرتوانایی‌های معماری‌های میکروسرویس، رویدادمحور و مبتنی بر فضا هستند. این دو ابرتوانایی پایگاه‌داده‌های کلید-مقدار و ستونی نیز هستند؛ یعنی این انواع پایگاه‌داده گزینه‌های خوبی برای تقویت آن ویژگی‌های معماری‌اند.

ساختار داده

ساختار داده‌ای که ذخیره و به آن دسترسی می‌شود نیز در این تقاطع عاملی تعیین‌کننده است. اگر **ساختار داده** رابطه‌ای باشد، یعنی بر سلسله‌مراتبی از روابط وابسته به هم بنا شده باشد، پایگاه‌دادهٔ رابطه‌ای به‌خوبی با آن همسو خواهد بود. اما ذخیرهٔ جفت‌های کلید-مقدار در یک پایگاه‌دادهٔ رابطه‌ای ناهمسویی‌ای است که می‌تواند هم در پایگاه‌داده و هم در معماری به ناکارآمدی بینجامد. همهٔ داده‌ها ساختار یکسانی ندارند، پس حواستان به این موضوع باشد. برخی داده‌ها ممکن است رابطه‌ای باشند، برخی دیگر سندمحور (به‌ویژه هنگام ذخیرهٔ محموله‌های رویداد یا درخواستِ مبتنی بر `JSON`) و برخی دیگر نیز کلید-مقدارمحور. با توجه به تنوع بالقوهٔ ساختارهای داده در هر معماری، توصیه می‌کنیم هر جا شدنی است از پایگاه‌داده‌های چندگانه `(polyglot databases)` بهره بگیرید.

اولویت خواندن/نوشتن

اگر مسئلهٔ کسب‌وکار شامل حجم بالای خواندن یا نوشتن باشد، این اطلاعات برای همسو کردنِ توپولوژی پایگاه‌داده با معماری اهمیت دارد. برای مثال، اگر معماری حجم بالای نوشتن را بر خواندن‌های گاه‌به‌گاه مقدم بداند، پایگاه‌دادهٔ ستونی گزینهٔ مناسبی خواهد بود. اما اگر عکس این صادق باشد و حجم بالای خواندن در اولویت باشد، پایگاه‌دادهٔ کلید-مقدار، سندمحور یا گرافی مناسب‌تر است. اگر سیستم خواندن و نوشتن را تقریباً به یک اندازه در اولویت قرار دهد، پایگاه‌داده‌های رابطه‌ای و `NewSQL` گزینه‌های خوبی‌اند. ناهمسوییِ این عامل می‌تواند به سیستم‌هایی با کارایی ضعیف بینجامد.

معماری و رویه‌های مهندسی

در اواخر سدهٔ بیستم، ده‌ها روش‌شناسی توسعهٔ نرم‌افزار رواج یافت، از جمله روش آبشاری و گونه‌های بسیاری از روش‌های چابک (مانند `Scrum`، `Extreme Programming`، `Lean` و `Crystal`). در آن زمان، بیشتر معماران بر این باور بودند که هیچ‌یک از این‌ها بر معماری نرم‌افزار اثری ندارد و توسعه را فرایندی یکسره جداگانه می‌دانستند. اما در چند سال گذشته، پیشرفت‌های مهندسی دغدغه‌های فرایندی را بر معماری نرم‌افزار تحمیل کرده‌اند. سودمند است که

فرایندهای توسعهٔ نرم‌افزار را از **رویه‌های مهندسی** جدا کنیم. منظور ما از **فرایندها** (processes) این است که تیم‌ها چگونه شکل می‌گیرند و مدیریت می‌شوند، جلسه‌ها چگونه برگزار می‌شوند و گردش‌های کار چگونه سازمان می‌یابند؛ به‌اختصار، سازوکار سازمان‌دهی و تعامل افراد. در مقابل، **رویه‌های مهندسی** (engineering practices) به فنون و ابزارهای مستقل از فرایندی اشاره دارند که تیم‌ها برای توسعه و انتشار نرم‌افزار به کار می‌برند. برای مثال، Extreme Programming (XP)، یکپارچه‌سازی مداوم (CI)، تحویل مداوم (continuous delivery یا CD) و توسعهٔ آزمون‌محور (TDD) همگی رویه‌های مهندسیِ اثبات‌شده‌ای‌اند که به فرایند خاصی وابسته نیستند. بنابراین، اصطلاح **مهندسی نرم‌افزار** (software engineering) هم توسعهٔ نرم‌افزار و هم این رویه‌ها را در بر می‌گیرد.

تمرکز بر رویه‌های مهندسی اهمیت دارد. توسعهٔ نرم‌افزار بسیاری از ویژگی‌های رشته‌های مهندسیِ بالغ‌تر را ندارد. برای مثال، مهندسان عمران می‌توانند تغییرات سازه‌ای را با دقتی بسیار بیشتر از دقت مهندسان نرم‌افزار در پیش‌بینی جنبه‌های مشابهِ ساختار نرم‌افزار پیش‌بینی کنند. این یعنی پاشنهٔ آشیل توسعهٔ نرم‌افزار **برآورد** (estimation) است: چقدر زمان، چه تعداد منابع، چقدر پول؟ بخشی از آنچه به این دشواری دامن می‌زند این است که رویه‌های سنتیِ برآورد با ماهیت اکتشافی توسعهٔ نرم‌افزار و ناشناخته‌هایی که معمولاً هنگام توسعهٔ نرم‌افزار پدید می‌آیند سازگار نیستند.

هرچند فرایند عمدتاً از معماری جداست، فرایندهای تکرارشونده (iterative) با ماهیت معماری سازگارترند. تلاش برای ساختن یک سیستم مدرن مانند میکروسرویس‌ها با فرایندی کهنه مانند روش آبشاری اصطکاک فراوانی پدید می‌آورد. یکی از جنبه‌های معماری که روش‌شناسی‌های چابک در آن به‌راستی می‌درخشند، مهاجرت از یک سبک معماری به سبکی دیگر است. روش‌شناسی‌های چابک از چنین تغییراتی بهتر از فرایندهای برنامه‌ریزی‌محور پشتیبانی می‌کنند، زیرا حلقه‌های بازخورد فشرده‌ای دارند و فنونی مانند الگوی Strangler و کلیدهای قابلیت (feature toggles) را تشویق می‌کنند.

معماران اغلب رهبر فنی پروژه نیز هستند و این یعنی تعیین اینکه تیم از چه رویه‌های مهندسی‌ای استفاده کند. همان‌طور که معماران پیش از انتخاب معماری حوزهٔ مسئله را با دقت بررسی می‌کنند، باید اطمینان یابند که سبک معماری و رویه‌های مهندسی‌شان با هم جور است. برای مثال، فلسفهٔ معماری میکروسرویس‌ها فرض می‌گیرد که تیم‌ها کارهایی مانند تأمین ماشین (machine provisioning)، آزمون و استقرار را خودکار می‌کنند. تلاش برای ساختن معماری میکروسرویس‌ها با یک گروه عملیاتِ کهنه، فرایندهای دستی و آزمونِ اندک به احتمال زیاد به شکست می‌انجامد. همان‌طور که حوزه‌های مسئلهٔ گوناگون به سوی سبک‌های معماری خاصی گرایش دارند، رویه‌های مهندسیِ گوناگون نیز چنین‌اند.

سیر تکامل اندیشه، از Extreme Programming تا تحویل مداوم و فراتر از آن، ادامه دارد؛ زیرا پیشرفت در رویه‌های مهندسی قابلیت‌های معماریِ تازه‌ای را ممکن می‌کند. کتاب Neal با عنوان **Building Evolutionary Architectures** شیوه‌های تازه‌ای برای اندیشیدن به تقاطع رویه‌های مهندسی و معماری را برجسته می‌کند که می‌توانند چگونگی خودکارسازی حاکمیت معماری را بهبود بخشند. این کتاب واژگان و شیوهٔ اندیشیدنِ تازه و مهمی دربارهٔ ویژگی‌های معماری ارائه می‌کند و فنونی را برای ساختن معماری‌هایی پوشش می‌دهد که در گذر زمان به‌نرمی تغییر می‌کنند.

در دنیای توسعهٔ نرم‌افزار، هیچ چیز ایستا نمی‌ماند. معماران ممکن است سیستمی را برای برآوردن معیارهای مشخصی طراحی کنند، اما برای اطمینان از اینکه طراحی‌هایشان هم از پیاده‌سازی و هم از روند ناگزیرِ تغییر جان سالم به در ببرد، آنچه نیاز داریم یک **معماری تکامل‌پذیر** `(evolutionary architecture)` است.

کتاب `Building Evolutionary Architectures` مفهوم استفاده از **توابع برازش معماری** `(architectural fitness functions)` را برای محافظت از ویژگی‌های معماری (و حاکمیت بر آن‌ها) در گذر زمان و با رخ دادن تغییرات معرفی می‌کند. از فصل ۶ به یاد دارید که توابع برازش معماری راهی برای به دست آوردن ارزیابی عینیِ یکپارچگیِ یک یا چند ویژگی معماری‌اند. این ارزیابی ممکن است سازوکارهای گوناگونی را در بر بگیرد، مانند سنجه‌ها، آزمون‌های واحد، پایشگرها و مهندسی آشوب.

برای دیدن اینکه چگونه می‌توان از توابع برازش برای کمک به همسو کردنِ این تقاطع استفاده کرد، مسئلهٔ کسب‌وکاریِ نیاز به زمان عرضهٔ سریع به بازار را در نظر بگیرید. زمان عرضه به بازار به **چابکی** `(agility)` ترجمه می‌شود: توانایی سیستم در پاسخ سریع به تغییر. چابکی یک ویژگی معماری مرکب است که از نگهداری‌پذیری، آزمون‌پذیری و استقرارپذیری تشکیل می‌شود (فصل ۶ را ببینید). هر سه ویژگی معماری از رویه‌ها و روال‌های مهندسی اثر می‌پذیرند و از این رو می‌توان آن‌ها را از راه توابع برازش سنجید و دنبال کرد. برای مثال، هم معماری میکروسرویس‌ها و هم معماری سرویس‌محور از سطوح بالای چابکی پشتیبانی می‌کنند. اما اگر رویه‌های مهندسیِ پیرامون این ویژگی‌های معماری با معماری همسو نباشند، سیستم به آن اهداف و نیازمندی‌های چابکی نخواهد رسید. توابع برازش می‌توانند به شناسایی ناهمسویی کمک کنند و معمار را برانگیزند تا برای همسو کردنِ دوبارهٔ رویه‌های مهندسی با معماری (یا برعکس) دست به کار شود.

معماری و توپولوژی‌های تیم

چنان‌که در سراسر بخش دوم کتاب بررسی کردیم، توپولوژی‌های تیم می‌توانند اثر مستقیمی بر معماری نرم‌افزار داشته باشند و برعکس. این همسویی چنان مهم است که برای هر سبک معماری‌ای که در این کتاب معرفی می‌کنیم، بخشی دربارهٔ توپولوژی‌های تیم گنجانده‌ایم.

یکی از پایه‌ای‌ترین راه‌های همسو شدنِ توپولوژی‌های تیم با معماری، نوع افراز است. تیم‌ها نیز مانند معماری‌ها می‌توانند به‌صورت حوزه‌ای یا به‌صورت فنی افراز شوند. تیم‌های افرازشده به‌صورت حوزه‌ای بر پایهٔ حوزه سازمان می‌یابند و معمولاً چندتخصصی‌اند و تخصص‌های گوناگون در سراسر تیم پخش است. برای مثال، یک تیم حوزه‌ای خاص ممکن است بر بخش مشتری‌محورِ یک سیستم تمرکز کند و از این رو مسئول پردازش سرتاسریِ کارکردهای مربوط به مشتری، از رابط کاربری تا پایگاه‌داده، باشد. در مقابل، تیم‌های افرازشده به‌صورت فنی هر یک بر یک کارکرد فنیِ مشخص از معماری تمرکز می‌کنند و معمولاً بر پایهٔ دسته‌های فنی سازمان می‌یابند: برای مثال، تیم‌های رابط کاربری، تیم‌های پردازش بک‌اند، تیم‌های سرویس‌های مشترک و تیم‌های پایگاه‌داده به‌خوبی با سبک معماری لایه‌ای همسو می‌شوند. همچنین ممکن است این تیم‌ها به‌صورت فنی به تیم‌های کارکرد کسب‌وکار و تیم‌های هم‌گام‌سازیِ داده افراز شوند که به‌خوبی با سبک معماری مبتنی بر فضا همسوست.

درک چگونگی سازمان‌دهی تیم‌ها برای تضمین موفقیت سیستم حیاتی است. اگر توپولوژی تیم‌های سازمان با معماری ناهمسو باشد، تیم‌ها در پیاده‌سازی و نگهداری معماری به دشواری می‌افتند و بعید است معماری به اهداف کسب‌وکاری‌اش برسد.

معماری و یکپارچه‌سازی سیستم‌ها

سیستم‌ها به‌ندرت در انزوا زندگی می‌کنند. بیشترشان به پردازش و دادهٔ بیشتری از سیستم‌های دیگر نیاز دارند؛ و این ما را به تقاطع معماری و یکپارچه‌سازی سیستم‌ها می‌رساند. وقتی یک سیستم باید برای انجام پردازش بیشتر یا بازیابی داده با سیستمی دیگر ارتباط برقرار کند، معمار آن با انبوهی از چالش‌ها و پیامدها روبه‌رو می‌شود. برای مثال، آیا سیستمی که فراخوانی می‌شود در دسترس است؟ آیا در مقیاس‌پذیری و کارایی هم‌سطحِ نیازمندی‌های سیستم فراخواننده است؟

وقتی معماران به اندازهٔ کافی بر یکپارچه‌سازی سیستم‌ها تمرکز نمی‌کنند، کوپلینگ ایستا و پویای سیستم‌ها اغلب به معماری‌هایی می‌انجامد که مقیاس‌پذیر نیستند، پاسخ‌گو نیستند و چابکی ندارند. هنگام یکپارچه‌سازی با سیستم‌های دیگر، در نظر بگیرید که از کدام پروتکل‌های ارتباطی استفاده کنید، چه نوع قراردادهایی میان سیستم‌ها داشته باشید، آیا ویژگی‌های معماری سیستم‌ها با هم سازگارند، و آیا یکپارچه‌سازی کوانتوم معماریِ هر سیستم را حفظ می‌کند یا نه.

معماری و سازمان

هر سازمانی مجموعه‌ای از استانداردها و اصول راهنما دارد. منظور ما از **سازمان** `(enterprise)` مجموعهٔ همهٔ سیستم‌ها و محصولات درون یک شرکت (یا یک بخش یا واحد درون شرکت) است. برای مثال، بسیاری از شرکت‌ها، فارغ از نوع سیستم، استانداردها، رویه‌ها یا روال‌های امنیتیِ مشخصی را بر راه‌حل‌های معماری تحمیل می‌کنند. استانداردهای سازمانی همچنین می‌توانند سکوها، فناوری‌ها، استانداردهای مستندسازی و استانداردهای نمودارکشی و مانند آن را در بر بگیرند. از استانداردها و رویه‌های سطح سازمان آگاه باشید و اطمینان یابید که معماری به‌درستی با آن‌ها همسوست.

ما موقعیت‌های بسیاری را تجربه کرده‌ایم که معمار رویه‌ها، استانداردها و روال‌های سطح سازمان را نادیده گرفته است. نتیجهٔ معمول این بوده که راه‌حل معماری، هر قدر هم از نظر فنی اثربخش، یک راه‌حلِ «یک‌باره»ی شکست‌خورده شمرده شده و کنار گذاشته شده است. هرچه بر اهمیت همسو کردنِ معماری با رویه‌های سازمان برای تضمین موفقیت آن تأکید کنیم، باز هم کم است.

معماری و محیط کسب‌وکار

محیط کسب‌وکار اثری چشمگیر و **مستقیم** بر معماری سیستم‌هایش دارد (و برعکس)، و محیط کسب‌وکار هرگز از تغییر باز نمی‌ایستد. آیا شرکت برای سرپا ماندن تدابیر سخت‌گیرانهٔ کاهش هزینه را از سر می‌گذراند، یا با شتاب در حال گسترش است؟ آیا کسب‌وکار هر فصل تغییر جهت می‌دهد و جایگاه خود را بازتعریف می‌کند تا در بازاری بسیار پرنوسان و رقابتی جای خود را بیابد، یا در وضعیتی پایدار قرار دارد؟ معمار نرم‌افزارِ اثربخش جایگاه و جهت‌گیری شرکت را درک می‌کند و معماری سیستم‌های حیاتی را با محیط کسب‌وکار همسو می‌کند.

ما این همسویی را **هم‌ریختی حوزه و معماری** `(domain-to-architecture isomorphism)` می‌نامیم. برای مثال، شرکت‌هایی که تدابیر شدید کاهش هزینه را از سر می‌گذرانند، با معماری‌های میکروسرویس یا مبتنی بر فضا که ساخت و نگهداری‌شان بسیار پرهزینه است، همسوییِ خوبی نخواهند داشت. در مقابل، کسب‌وکارهایی که از راه ادغام و تملک با شتاب گسترش می‌یابند، از سبک‌های معماری تک‌پارچه که توانایی تکامل و انطباق ندارند، به‌خوبی بهره‌مند نمی‌شوند.

یکی از مسائلی که معماران معمولاً در این تقاطع با آن روبه‌رو می‌شوند، تغییر کسب‌وکار است؛ به‌ویژه **تغییر ناشناخته** `(unknown change)`. `Donald Rumsfeld`، وزیر دفاع پیشین ایالات متحده، زمانی جملهٔ مشهوری گفت:

> «دانسته‌های دانسته‌ای هست؛ چیزهایی که می‌دانیم که می‌دانیم. همچنین می‌دانیم که نادانسته‌های دانسته‌ای هست؛ یعنی می‌دانیم چیزهایی هست که نمی‌دانیم. اما نادانسته‌های نادانسته‌ای نیز هست؛ چیزهایی که نمی‌دانیم که نمی‌دانیم.»

بسیاری از محصولات و سیستم‌ها با فهرستی از **نادانسته‌های دانسته** `(known unknowns)` آغاز می‌شوند: چیزهایی دربارهٔ حوزه و فناوری که توسعه‌دهندگان باید بیاموزند و می‌دانند تغییر خواهند کرد. اما همین سیستم‌ها قربانی **نادانسته‌های نادانسته** `(unknown unknowns)` نیز می‌شوند: چیزهایی که هیچ‌کس نمی‌دانست قرار است سر برآورند، اما به‌طور غیرمنتظره پدیدار شده‌اند. «نادانسته‌های نادانسته» دشمنان سرسخت سیستم‌های نرم‌افزاری‌اند. به همین دلیل است که همهٔ تلاش‌های نرم‌افزاری مبتنی بر «طراحی بزرگ از پیش» `(Big Design Up Front)` آسیب می‌بینند: معماران نمی‌توانند برای نادانسته‌های نادانسته طراحی کنند. به نقل از `Mark`:

> «همهٔ معماری‌ها به سبب **نادانسته‌های نادانسته** تکرارشونده می‌شوند. چابک فقط این را می‌پذیرد و زودتر انجامش می‌دهد.»

برنامه‌ریزی برای تغییر در معماری نرم‌افزار **دشوار** است. رویه‌های معماری تکامل‌پذیر، همانند معماری تکرارشونده، به رویارویی با چشم‌انداز همواره در حال تغییرِ کسب‌وکار کمک می‌کنند. پذیرفتن ویژگی‌های معماری‌ای مانند **قابلیت حمل**، **مقیاس‌پذیری**، **تکامل‌پذیری** و **انطباق‌پذیری** نیز به انعطاف‌پذیرتر و سازگارپذیرتر شدنِ معماری نرم‌افزار در برابر تغییر کمک می‌کند.

Barry O’Reilly، معمار نرم‌افزار باتجربه‌ای که در نظریهٔ پیچیدگی (complexity theory) و طراحی نرم‌افزار تخصص دارد، شیوهٔ تازه‌ای برای اندیشیدن به تغییر پیوستهٔ کسب‌وکار پدید آورده است که **نظریهٔ باقی‌ماندگی** (residuality theory) نام دارد. در کتاب خود O’Reilly **Residues: Time, Change, and Uncertainty in Software Architecture** (Leanpub، ۲۰۲۴) فنونی را شرح می‌دهد که در آن‌ها تغییر کسب‌وکار به‌عنوان **عامل فشار** (stressor) و تغییرات معماریِ متناظر با آن به‌عنوان **باقی‌مانده** (residue) در نظر گرفته می‌شوند. نظریهٔ او این است که هرچه معمار با افزودن باقی‌مانده‌های بیشتر به معماری به تغییر پاسخ دهد، این باقی‌مانده‌ها سرانجام به تغییرات **ناشناخته‌ای** نیز پاسخ خواهند داد که معمار به هیچ روی نمی‌تواند پیش‌بینی‌شان کند؛ و معماری‌ای پدید می‌آید که در چارچوب نظریهٔ پیچیدگی به حالتی بحرانی رسیده است. این نظریهٔ جالبی است که ما از نزدیک دنبالش می‌کنیم.

معماری و هوش مصنوعی مولد

اکنون که ویراست دوم این کتاب را در اوایل ۲۰۲۵ می‌نویسیم، **هوش مصنوعی مولد** (generative artificial intelligence یا Gen AI) و مدل‌های زبانی بزرگ (LLMs) به دنیای توسعه و طراحی نرم‌افزار راه یافته‌اند. بسیاری از شرکت‌ها آن‌ها را در سیستم‌ها به کار می‌گیرند تا کارهایی را انجام دهند که پیش‌تر تنها انسان‌ها به‌صورت دستی انجام می‌دادند. شگفت‌آور نیست که هوش مصنوعی مولد با معماری نرم‌افزار نیز تقاطع دارد: معماران مدل‌های زبانی بزرگ را در معماری‌های نرم‌افزار می‌گنجانند و برخی حتی از ابزارهای هوش مصنوعی مولد کمک می‌گیرند تا در مسائل دشوار بیندیشند.

گنجاندن هوش مصنوعی مولد در معماری

یکی از رویکردهایی که برای گنجاندن هوش مصنوعی مولد در معماری توصیه می‌کنیم، بهره‌گیری از **انتزاع** (abstraction) و **ماژولاریتی** است. مهم است که بتوان یک مدل زبانی بزرگ را به‌سرعت با مدلی دیگر جایگزین کرد، و امکانِ نرده‌های محافظ (guardrails یا rails) و ارزیابی نتایج (evaluations یا evals) مدل‌های زبانی گوناگون را فراهم کرد.

برای مثال، فرض کنید یک شرکت کاریابی می‌خواهد از هوش مصنوعی مولد برای ناشناس‌سازی (anonymize) رزومه‌ها بهره بگیرد، با این هدف که سوگیری کاهش یابد و تمرکز بر مهارت‌های کارجویان باشد، نه بر ویژگی‌های جمعیت‌شناختی یا عواملی از این دست. این کار معمولاً به دست انسان‌ها انجام می‌شود، اما یک مدل زبانی بزرگ نیز به‌آسانی از عهده‌اش برمی‌آید. اما آیا نتایج مدل زبانی دقیق است؟ آیا اطلاعات بیش از حدی را از رزومه حذف می‌کند؟ آیا اطلاعات جمعیت‌شناختیِ بیش از حدی را باقی می‌گذارد؟ برای این نوع سیستم، حیاتی است که بتوان نمونه‌ها و سنجه‌ها را گرد آورد و موتورهای مدل زبانی را با هم مقایسه کرد. ابزارهایی مانند Langfuse به ایجاد این نوع مشاهده‌پذیری در معماری کمک می‌کنند.

هوش مصنوعی مولد به‌عنوان دستیار معمار

یک مدل زبانی بزرگ (مانند `Copilot`) با پرامپت مناسب می‌تواند کد منبعی تولید کند که در زمان و تلاش توسعه‌دهندگان صرفه‌جویی فراوانی می‌کند. این مدل‌ها برای حل مسائل بسیار مشخص و قطعی عالی‌اند؛ مانند «به زبان برنامه‌نویسی `C#` کد منبعی بنویس که یک `PIN` چهاررقمیِ یکتا بدون ارقام تکراری تولید کند.» اما آیا فناوری مدل‌های زبانی بزرگ می‌تواند در کارهای معمول به معماران نرم‌افزار کمک کند؟ در ادامه چند نمونهٔ کلی از پرامپت‌های مرتبط با معماری آمده است:

- ارزیابی ریسک: «آیا در این معماری حوزه‌های پرریسک وجود دارد؟»
- کاهش ریسک: «با این ریسک چگونه برخورد کنم؟»
- ضدالگوها: «آیا ضدالگوی رایجی در این معماری وجود دارد؟»
- تصمیم‌ها: «برای این گردش کار از ارکستراسیون استفاده کنم یا کرئوگرافی؟»

تا زمان نگارش این ویراست دوم (اوایل ۲۰۲۵)، در این زمینه موفقیت چندانی نداشته‌ایم. پرسیدن از یک مدل زبانی بزرگ که برای موقعیتی مشخص معماری میکروسرویس‌ها مناسب‌تر است یا معماری مبتنی بر فضا، به‌ندرت (اگر اصلاً) پاسخ درست را به دست می‌دهد. چرا؟ زیرا چنان‌که در این کتاب نشان داده‌ایم، **همه‌چیز در معماری نرم‌افزار یک مصالحه است**. مدل‌های زبانی بزرگ برای درک **دانش** عالی‌اند، اما تا امروز همچنان **خردِ** لازم برای گرفتن تصمیم‌های مناسب را ندارند. این خرد چنان بافتار گسترده‌ای را در بر می‌گیرد که برای معمار بسیار سریع‌تر است خودش یک مسئلهٔ کسب‌وکار را حل کند تا اینکه همه‌چیز را دربارهٔ مسئله و محیط و بافتار گسترده‌اش به یک مدل زبانی بیاموزد. همین که هشت تقاطع دیگر را برای توجه برشمرده‌ایم، باید گواه کافی بر دشواری این کار باشد.

با این همه، ابزارهای امیدوارکننده‌ای نیز دیده‌ایم. برای نمونه، `Thoughtworks Haiven` می‌تواند یک نمودار معماری را تفسیر کند و معماری نرم‌افزار را به‌طور کامل توصیف کند؛ و به این ترتیب کارِ صادر کردنِ نمودار به یک قالب ماشین‌خوان `(machine-readable format)` مانند `XML` و استفاده از آن در پرامپت مدل زبانی را حذف می‌کند. کاربران پس از وارد کردنِ این اطلاعات می‌توانند پرسش‌های ساده‌ای دربارهٔ معماری از `Haiven` بپرسند؛ مثلاً اینکه آیا می‌تواند گلوگاه‌ها یا مشکلاتی را شناسایی کند. تلاش‌های دیگر شامل استفاده از یک مدل زبانی بزرگ برای ترجمهٔ یک نمودار `PlantUML` یا توصیفی شبه‌زبانی از معماری به کد اجرایی `ArchUnit` برای حاکمیت بر ساختار سیستم بوده است. فعالیت زیادی در این زمینه در جریان است، پس در سال‌های آینده انتظار تغییرات سریعی را در چگونگی کمک هوش مصنوعی مولد به معماران داشته باشید.

خلاصه

معماری نرم‌افزار فعالیتی کل‌نگر است که جنبه‌های بسیاری از یک سازمان را در بر می‌گیرد. معماران نرم‌افزارِ اثربخش درمی‌یابند که طراحی و نگهداری یک معماری بسیار فراتر از صرفاً انتخاب یک سبک معماری خاص و پیش رفتن به سوی پیاده‌سازی است. این کار همچنین به معنای اطمینان از همسوییِ معماری با دیگر جنبه‌های محیط، و بهره‌گیری از مهارت‌های ارتباط و همکاری‌ای است که در بخش سوم شرح داده شد تا این همسویی محقق شود.

واژه‌نامهٔ اختصاصی این فصل

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Intersections of Architecture	تقاطع‌های معماری
Implementation	پیاده‌سازی
Infrastructure	زیرساخت
Data Topologies	توپولوژی‌های داده
Engineering Practices	رویه‌های مهندسی
Team Topologies	توپولوژی‌های تیم
Systems Integration	یکپارچه‌سازی سیستم‌ها
The Enterprise	سازمان
Business Environment	محیط کسب‌وکار
Operational Concerns	دغدغه‌های عملیاتی
In-Memory Replicated Cache	کش تکثیرشدهٔ درون‌حافظه‌ای
Internal Memory	حافظهٔ داخلی
Out-of-Memory	کمبود حافظه
Logical Architecture	معماری منطقی
Operations	عملیات
Elastic Scale	مقیاس‌پذیری کشسان
Region (cloud)	منطقه
Availability Zone	ناحیهٔ دسترس‌پذیری
Co-locating	هم‌مکان‌سازی
Superpowers (architecture)	ابرتوانایی‌ها
Key-Value Database	پایگاه‌دادهٔ کلید-مقدار
Columnar Database	پایگاه‌دادهٔ ستونی

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Graph Database	پایگاه‌دادهٔ گرافی
Polyglot Databases	پایگاه‌داده‌های چندگانه
Processes (development)	فرایندها
Continuous Delivery (CD)	تحویل مداوم (CD)
Software Engineering	مهندسی نرم‌افزار
Estimation	برآورد
Iterative Process	فرایند تکرارشونده
Feature Toggles	کلیدهای قابلیت
Machine Provisioning	تأمین ماشین
Domain-to-Architecture Isomorphism	هم‌ریختی حوزه و معماری
Unknown Change	تغییر ناشناخته
Known Unknowns	نادانسته‌های دانسته
Unknown Unknowns	نادانسته‌های نادانسته
Big Design Up Front	طراحی بزرگ از پیش
Complexity Theory	نظریهٔ پیچیدگی
Residuality Theory	نظریهٔ باقی‌ماندگی
Stressor	عامل فشار
Residue	باقی‌مانده
Abstraction	انتزاع
Guardrails	نرده‌های محافظ
Evals (evaluations)	ارزیابی‌ها
Anonymize	ناشناس‌سازی
Machine-Readable Format	قالب ماشین‌خوان